回溯法综合题题库（理论分析型）

题型说明

回溯法综合题的典型作答要求：

1. **定义解空间**：明确解空间的结构（子集树/排列树/满m叉树）
2. **确定约束条件**：可行性剪枝条件
3. **确定限界函数**：最优性剪枝条件（针对优化问题）
4. **画出解空间树**：标明搜索路径和剪枝点
5. **伪代码实现**：DFS + 剪枝
6. **复杂度分析**：最坏情况O(2^n)或O(n!)或O(m^n)

题目0：子集和问题（回溯入门案例）

【问题描述】 给定一个正整数集合S = {a_1, a_2, ..., a_n}和一个目标值T。判断是否存在S的一个子集，其元素之和等于T。

【输入示例】 S = {5, 10, 12, 13, 15, 18}，T = 30

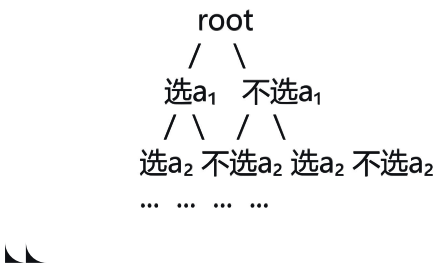
【输出示例】 存在解：{5, 10, 15} 或 {12, 18}

【答题框架】

1. 解空间分析

- **解空间结构**：子集树（每个元素选或不选）
- **解空间大小**：2^n
- **树结构**：从根到叶子的每条路径对应一个子集，深度为n

text



2. 约束条件与剪枝

- **约束条件**：当前已选元素之和 ≤ T（超过T则剪枝）
- **上界剪枝**：若当前和 + 剩余所有元素之和 < T，则不可能达到T，剪枝

3. 伪代码

text

```
算法 SubsetSum(i, currentSum, totalRemaining)
// 输入：当前考虑第i个元素，当前和，剩余元素总和
// 输出：若找到解则返回true

if currentSum = T then
    print 当前已选元素
    return true
end if

if i > n then
    return false
end if

// 剪枝：当前和已超过目标
if currentSum > T then
    return false
end if

// 剪枝：即使选上所有剩余元素也达不到目标
if currentSum + totalRemaining < T then
    return false
end if

// 选第i个元素
selected[i] ← true
if SubsetSum(i+1, currentSum + a[i], totalRemaining - a[i]) then
    return true
end if

// 不选第i个元素
selected[i] ← false
if SubsetSum(i+1, currentSum, totalRemaining - a[i]) then
    return true
end if

return false
```

4. 时间复杂度分析

- 最坏情况：遍历整个子集树，**$O(2^n)$**
- 最好情况：剪枝提前终止
- 空间复杂度：**$O(n)$**（递归栈深度）

题目1：n皇后问题

【问题描述】 在n×n的棋盘上放置n个皇后，使得任意两个皇后都不在同一行、同一列或同一对角线上。请设计回溯算法找出所有合法的放置方案。

【输入示例】 n = 4

【输出示例】 2个解

text

```
解1：皇后位置：(1,2), (2,4), (3,1), (4,3)
解2：皇后位置：(1,3), (2,1), (3,4), (4,2)
```

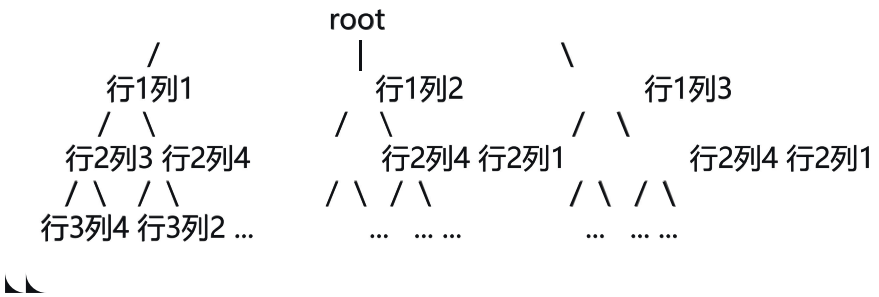
【答题框架】

1. 解空间分析

- **解空间结构**：排列树（每行放一个皇后，且列号互不相同）
- **解空间大小**：n!（实际因对角线约束会更少）
- **表示方法**：用数组col[i]表示第i行皇后所在的列号，col[1..n]是1..n的一个排列

解空间树示例（n=4）：

text



2. 约束条件

设第i行皇后放在第j列（即col[i]=j），对于已放置的k行（1 ≤ k < i）：

- **列约束**：col[k] ≠ j（由排列树结构保证，无需额外检查）
- **对角线约束**：|i - k| ≠ |j - col[k]|
  - 主对角线（左上到右下）：行差 = 列差，即 j - col[k] = i - k
  - 副对角线（右上到左下）：行差 = -列差，即 j - col[k] = -(i - k)

约束条件合并：
col[k] ≠ j 且 |i - k| ≠ |j - col[k]|

3. 伪代码

text

```
算法 nQueens(n)
// 输出：所有合法解

1. col[1..n] ← 0          // col[i]表示第i行皇后的列号
2. count ← 0              // 解的数量
```

```
3. call Backtrack(1)
4. return count
```

text

```
算法 Backtrack(row)
// 输入：当前要放置的行号row

1. if row > n then
2.     count ← count + 1
3.     输出解 col[1..n]
4.     return
5. end if
6.
7. for col[row] ← 1 to n do
8.     if isSafe(row) then
9.         Backtrack(row + 1)
10.    end if
11. end for
```

text

```
算法 isSafe(row)
// 检查第row行放在col[row]列是否与前面冲突

1. for i ← 1 to row-1 do
2.     if col[i] = col[row] then
3.         return false
4.     end if
5.     if |i - row| = |col[i] - col[row]| then
6.         return false
7.     end if
8. end for
9. return true
```

4. 优化版本（用数组记录冲突）

为了提高效率，可用三个布尔数组记录列和对角线的占用情况：

text

```
算法 nQueens_Optimized(n)

1. col[1..n] ← 0
2. // 冲突标记数组
3. columnUsed[1..n] ← false     // 列是否被占用
4. diag1Used[1..2n-1] ← false   // 主对角线（/方向）：i+j 为常数
5. diag2Used[1..2n-1] ← false   // 副对角线（\方向）：i-j+n 为常数
6. count ← 0
7. call Backtrack(1)
8. return count
```

```
text
```

```
算法 Backtrack(row)

1.  if row > n then
2.      count ← count + 1
3.      输出解
4.      return
5.  end if
6.
7.  for j ← 1 to n do
8.      if not columnUsed[j] and
9.         not diag1Used[row+j] and
10.        not diag2Used[row-j+n] then
11.
12.         col[row] ← j
13.         columnUsed[j] ← true
14.         diag1Used[row+j] ← true
15.         diag2Used[row-j+n] ← true
16.
17.         Backtrack(row + 1)
18.
19.         // 回溯，撤销标记
20.         columnUsed[j] ← false
21.         diag1Used[row+j] ← false
22.         diag2Used[row-j+n] ← false
23.     end if
24. end for
```

5. 时间复杂度分析

- 最坏情况：需要遍历整个排列树，**O(n!)**
- 实际运行受约束剪枝影响：
  - n=4时：递归调用次数远小于4!=24
  - n=8时：仅需约15720次检查
- 空间复杂度：**O(n)**（col数组 + 递归栈 + 标记数组）

n皇后问题解的数量（前几项）：

n	解的数量
4	2
5	10
6	4
7	40
8	92
9	352
10	724

题目2：地图着色问题

【问题描述】 给定一个无向图G = (V, E)和m种颜色，要求为每个顶点分配一种颜色，使得相邻顶点颜色不同。判断是否存在合法的着色方案，并找出所有方案。

【输入示例】 图结构：

text

```
顶点数 n = 5
边集：(1,2), (1,3), (2,3), (2,4), (3,4), (3,5), (4,5)
颜色数 m = 3
```



（这是一个5个顶点的图，类似一个带对角线的五边形）

【输出示例】 存在多种着色方案，例如：
方案1：1:红, 2:绿, 3:蓝, 4:红, 5:绿
方案2：...

【答题框架】

1. 解空间分析

- **解空间结构**：满m叉树（每个顶点有m种颜色可选）
- **解空间大小**：m^n（不考虑约束）
- **树结构**：深度为n，每个内部节点有m个分支

text

```
                root
      /      |      \      ... (m个分支)
   颜色1   颜色2   颜色3   ...  (顶点1可选颜色)
   /|\     /|\     /|\
颜色1... 颜色1... 颜色1...  (顶点2可选颜色)
(每个分支继续扩展，直到顶点n)
```



2. 约束条件

设color[i]表示顶点i的颜色编号（1 ≤ color[i] ≤ m）。

对于每条边(i, j) ∈ E，必须满足：
color[i] ≠ color[j]

在回溯过程中，当为顶点i选择颜色c时，只需检查与所有已着色邻接点是否冲突。

3. 伪代码

text

```
算法 MapColoring(G, m)
// 输入：图G，颜色数m
// 输出：所有合法着色方案

1. n ← |V|
2. color[1..n] ← 0        // 0表示未着色
3. count ← 0              // 方案计数
4. call Backtrack(1)
5. return count
```

text

```
算法 Backtrack(i)
// 输入：当前要着色的顶点编号i

1. if i > n then
2.     count ← count + 1
3.     输出颜色方案 color[1..n]
4.     return
5. end if
6.
7. for c ← 1 to m do
8.     if isSafe(i, c) then
9.         color[i] ← c
10.        Backtrack(i + 1)
11.        color[i] ← 0    // 回溯，撤销着色
12.    end if
13. end for
```

text

```
算法 isSafe(i, c)
// 检查顶点i着颜色c是否与已着色邻接点冲突

1. for each neighbor j of i do
2.     if color[j] = c then
3.         return false
4.     end if
5. end for
6. return true
```

4. 优化策略

按度数排序：优先为度数大的顶点着色（称为MRV启发式，Minimum Remaining Values）。度数大的顶点约束更多，先着色可以尽早发现冲突，减少搜索空间。

text

```
算法 MapColoring_Optimized(G, m)

1. n ← |V|
2. 将顶点按度数降序排序，得到顺序order[1..n]
```

3. color[1..n] ← 0
4. count ← 0
5. call Backtrack(1, order)
6. return count

5. 四色定理

四色定理：任何平面图都可以用4种颜色着色，使得相邻区域颜色不同。这是一个著名的数学定理，但证明极其复杂（使用计算机辅助证明）。

推论：

- 对于平面图，m=4时总有解
- 对于一般图，判断能否用m种颜色着色是NP-完全问题

6. 时间复杂度分析

- 最坏情况：遍历整个m叉树，**$O(m^n)$**
- 实际运行受图结构和约束剪枝影响
- 对于稀疏图或平面图，搜索空间远小于m^n
- 空间复杂度：**O(n)**（color数组 + 递归栈）

题目3：0-1背包问题（回溯法）

【问题描述】 用回溯法解0-1背包问题。n个物品，容量C，重量w[i]，价值v[i]。要求输出最优解和最优值，并分析剪枝条件。

【输入示例】 n=4, C=7, w=[3,5,2,1], v=[9,10,7,4]

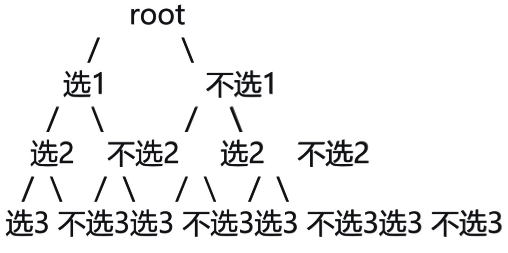
【输出示例】 最优值：20，最优解：[1,0,1,1]（选择物品1,3,4，总重3+2+1=6≤7，价值9+7+4=20）

【答题框架】

1. 解空间分析

- **解空间结构**：子集树（每个物品选或不选）
- **解空间大小**：2^n

text



2. 约束条件与限界函数

- **约束条件**（可行性剪枝）：当前总重量 ≤ C
- **限界函数**（最优性剪枝）：
  当前总价值 + 剩余物品的最大可能价值 ≤ 当前最优值

剩余物品最大可能价值的估算：

- 方法1：剩余物品价值之和（过于乐观，剪枝效率低）
- 方法2：按单位重量价值降序排序后，用贪心法估算上界（更精确）

3. 预处理（按单位重量价值排序）

为了获得更好的上界估计，通常先将物品按单位重量价值 v[i]/w[i] 降序排序。

text

```
算法 Knapsack_Backtrack(w, v, C, n)

1. 将物品按单位重量价值 v[i]/w[i] 降序排序
2. bestValue ← 0
3. bestSolution[1..n] ← 0
4. currentSolution[1..n] ← 0
5. call Backtrack(0, 0, 0)
6. return bestValue, bestSolution
```

4. 伪代码

text

```
算法 Backtrack(i, cw, cv)
// i: 当前考虑的物品下标（从0开始）
// cw: 当前总重量
// cv: 当前总价值

1. if i = n then
2.     if cv > bestValue then
3.         bestValue ← cv
4.         bestSolution ← currentSolution
5.     end if
6.     return
7. end if
8.
9. // 计算剩余物品的最大可能价值（上界）
10. if cv + bound(i, cw) ≤ bestValue then
11.     return                // 限界剪枝
12. end if
13.
14. // 不选第i个物品
15. currentSolution[i] ← 0
16. Backtrack(i+1, cw, cv)
17.
```

```
18. // 选第i个物品（约束剪枝）
19. if cw + w[i] ≤ C then
20.     currentSolution[i] ← 1
21.     Backtrack(i+1, cw + w[i], cv + v[i])
22. end if
```

text

```
算法 bound(i, cw)
// 计算剩余物品的最大可能价值（贪心上界）

1. 剩余容量 = C - cw
2. 上界 = 0
3. for j ← i+1 to n-1 do
4.      if w[j] ≤ 剩余容量 then
5.          上界 ← 上界 + v[j]
6.          剩余容量 ← 剩余容量 - w[j]
7.      else
8.          上界 ← 上界 + (v[j] / w[j]) * 剩余容量  // 取部分物品
9.          break
10.     end if
11. end for
12. return 上界
```

text

5. 时间复杂度分析

- 最坏情况：遍历整个子集树，**$O(2^n)$**
- 最好情况：加限界函数后可大幅剪枝
- 空间复杂度：**O(n)**（递归栈深度 + 记录数组）

题目4：旅行商问题（TSP）

【问题描述】 给定n个城市和城市之间的距离矩阵，求一条经过每个城市恰好一次并返回起点的最短回路（哈密顿回路）。

【输入示例】 4个城市，距离矩阵：

text

```
   1   2   3   4
1  0  10  15  20
2 10   0  35  25
3 15  35   0  30
4 20  25  30   0
```

【输出示例】 最短回路长度：80（路线：1→2→4→3→1 或 1→3→4→2→1）

【答题框架】

1. 解空间分析

- **解空间结构**：排列树（固定起点1，其余n-1个城市的排列）
- **解空间大小**：(n-1)!

2. 约束条件与限界函数

- **约束条件**：下一个访问城市未访问过
- **限界函数**：
  当前已走路径长度 + 剩余城市的最小可能路径 ≥ 当前最优值

剩余城市最小可能路径的估算：

- 方法1：取剩余顶点间最小边之和的下界
- 方法2：用最小生成树估算下界（更精确）

3. 伪代码

text

```
算法 TSP_Backtrack(dist, n)
// 输入：距离矩阵dist[1..n][1..n]
// 输出：最短回路长度和路径

1. bestLen ← ∞
2. bestPath[1..n+1] ← []
3. currentPath[1] ← 1        // 从城市1出发
4. visited[1] ← true
5. call Backtrack(2, 1, 0)   // step=2表示已确定第2个城市，cur=1当前位置，len=0
6. return bestLen, bestPath
```

text

```
算法 Backtrack(step, cur, curLen)
// step: 已确定第几个城市（从2开始）
// cur: 当前所在城市
// curLen: 当前已走路径长度

1. if step = n+1 then
2.     // 已访问所有城市，需要返回起点
3.     totalLen ← curLen + dist[cur][1]
4.     if totalLen < bestLen then
5.         bestLen ← totalLen
6.         bestPath ← currentPath
7.         bestPath[n+1] ← 1   // 最后返回起点
8.     end if
9.     return
10. end if
11.
12. for next ← 2 to n do
13.     if not visited[next] then
14.         newLen ← curLen + dist[cur][next]
15.
```

```
16.         // 限界剪枝
17.         if newLen < bestLen then
18.             visited[next] ← true
19.             currentPath[step] ← next
20.             Backtrack(step+1, next, newLen)
21.             visited[next] ← false
22.         end if
23.     end if
24. end for
```

4. 改进的下界估计函数

```
text

算法 lowerBound(step, curLen)
// 估计剩余路径的最小可能长度

1.  下界 ← curLen
2.
3.  // 从当前城市出发的最小出边
4.  下界 ← 下界 + min{ dist[cur][j] | j未访问 }
5.
6.  // 对于每个未访问城市，加上其最小出边
7.  for each 未访问城市 v do
8.      下界 ← 下界 + min{ dist[v][u] | u ≠ v 且 (u未访问或u=1) }
9.  end for
10.
11. return 下界
```

5. 时间复杂度分析

- 最坏情况：遍历整个排列树，**O((n-1)!)**
- 实际运行受限界函数剪枝影响
- 空间复杂度：**O(n)**（递归栈 + visited数组 + 路径记录）

回溯法复杂度总结表

问题	解空间结构	解空间大小	约束条件	时间复杂度	空间复杂度
子集和	子集树	2^n	和 ≤ T	$O(2^n)$	O(n)
n皇后	排列树	n!	不同列、不同对角线	O(n!)	O(n)
地图着色	满m叉树	m^n	邻接点颜色不同	$O(m^n)$	O(n)
0-1背包	子集树	2^n	重量 ≤ C + 限界	$O(2^n)$	O(n)
TSP	排列树	(n-1)!	不重复访问 + 限界	O((n-1)!)	O(n)

哈密顿回路问题

【问题描述】 给定一个无向图G = (V, E)，判断是否存在一条回路，经过每个顶点恰好一次并返回起点。若存在，找出任意一条哈密顿回路。

与TSP的区别：

- TSP是带权图，找最短回路（优化问题）
- 哈密顿回路是**无权图**（或单位权），只判断是否存在回路（判定/搜索问题）

【输入示例】 图结构（无向图）：

text

```
顶点数 n = 5
边集：(1,2), (1,3), (2,3), (2,4), (2,5), (3,4), (3,5), (4,5)
```

(这是一个完全图K_5去掉边(1,4)、(1,5)后的图)

【输出示例】 存在哈密顿回路，例如：1→2→4→5→3→1

【答题框架】

1. 解空间分析

- **解空间结构**：排列树（固定起点1，其余n-1个城市的排列）
- **解空间大小**：(n-1)!
- **与TSP的区别**：无权图，只需要检查边是否存在，不需要计算权重

解空间树示例（n=4）：

text

```
                        root(起点1)
          /                 |                 \
      路径:1→2            路径:1→3            路径:1→4
       /   \              /   \              /   \
   1→2→3  1→2→4       1→3→2  1→3→4       1→4→2  1→4→3
    |      |           |      |           |      |
1→2→3→4 1→2→4→3     1→3→2→4 1→3→4→2     1→4→2→3 1→4→3→2
  检查能否 检查能否      检查能否 检查能否      检查能否 检查能否
  返回起点 返回起点      返回起点 返回起点      返回起点 返回起点
```

2. 约束条件

设path[1..n+1]记录路径，path[1]=起点，path[n+1]=起点。

对于相邻顶点path[k]和path[k+1]（1 ≤ k ≤ n），必须满足：

- **存在边**：(path[k], path[k+1]) ∈ E
- **不重复访问**：path[k] ≠ path[m] 对于任意 k ≠ m（前n个顶点互不相同）

在回溯过程中，当为第step个位置选择顶点v时，需检查：

1. 边(path[step-1], v)是否存在
2. v是否已经被访问过

最后，当step = n+1时，还需检查最后一点是否能回到起点：边(path[n], 1)是否存在。

3. 伪代码

text

```
算法 HamiltonianCycle(G)
// 输入：无向图G = (V, E)，顶点编号1..n
// 输出：若存在哈密顿回路，输出一条回路，否则输出不存在

1.  n ← |V|
2.  path[1..n+1] ← 0          // 存储路径，path[n+1]需要回到起点
3.  visited[1..n] ← false
4.  path[1] ← 1               // 从顶点1开始
5.  visited[1] ← true
6.
7.  if Backtrack(2) then
8.      输出路径 path[1..n+1]
9.      return true
10. else
11.     print "不存在哈密顿回路"
12.     return false
13. end if
```

text

```
算法 Backtrack(step)
// 输入：当前要放置的顶点位置step（2 ≤ step ≤ n+1）
// 输出：若找到回路返回true，否则返回false

1.  if step = n+1 then
2.      // 所有顶点已放置，检查是否能回到起点
3.      if 边(path[n], 1)存在 then
4.          path[n+1] ← 1
```

```
5.          return true
6.      else
7.          return false
8.      end if
9.  end if
10.
11. for v ← 1 to n do
12.     if not visited[v] and 边(path[step-1], v)存在 then
13.         // 选择v作为下一个顶点
14.         path[step] ← v
15.         visited[v] ← true
16.
17.         if Backtrack(step+1) then
18.             return true
19.         end if
20.
21.         // 回溯，撤销选择
22.         visited[v] ← false
23.         path[step] ← 0
24.     end if
25. end for
26.
27. return false
```

4. 优化策略

剪枝优化:

1. **度数检查**：若某个顶点的度数 < 2，则不可能存在哈密顿回路（因为回路中每个顶点恰好有两个邻接点），可提前结束。
2. **提前检查连通性**：若图不连通，则不存在哈密顿回路。

text

```
算法 预检查()

1.  for each vertex v in V do
2.      if degree(v) < 2 then
3.          return false      // 度小于2，不可能有回路
4.      end if
5.  end for
6.
7.  if 图不连通 then
8.      return false
9.  end if
10.
11. return true
```

启发式优化：

- 优先选择度数小的顶点进行扩展（约束更强的顶点先处理，减少分支）

5. 与TSP的对比

特性	哈密顿回路	旅行商问题(TSP)
图类型	无权图（或有边/无边）	带权图
目标	判断是否存在回路	找最短回路
解空间	排列树	排列树
约束条件	边必须存在	任意两点都有边（完全图）
剪枝	不存在的边剪枝	路径长度剪枝
时间复杂度	O(n!)	O(n!)
应用场景	图论、排班问题	路径规划、物流配送

6. 时间复杂度分析

- 最坏情况：遍历整个排列树，**O(n!)**（当图是完全图时）
- 实际运行受图结构和约束剪枝影响
- 对于稀疏图，可在早期剪掉大量分支
- 空间复杂度：**O(n)**（path数组 + visited数组 + 递归栈）

回溯专题更新后的题目列表

序号	题目	解空间结构	复杂度	核心约束
0	子集和问题	子集树	$O(2^n)$	和不超过T
1	n皇后问题	排列树	O(n!)	不同列、不同对角线
2	地图着色	满m叉树	$O(m^n)$	邻接点颜色不同
3	0-1背包	子集树	$O(2^n)$	重量+限界函数
4	旅行商(TSP)	排列树	O((n-1)!)	边存在+路径长度剪枝

序号	题目	解空间结构	复杂度	核心约束
5	**哈密顿回路**	**排列树**	**O(n!)**	**边存在+不重复**